An Agentic LLM System for the Preliminary Structural Design of an Aircraft Wing Box

Rouzbeh Pourjafarian (404201984)

MSc Student, Sharif University of Technology

Term project, *Aircraft Structural Design*, instructor Dr. Haddadpour, teaching assistant Eng. Samiei

August 20, 2026

Abstract

This report presents the architecture and engineering basis of an agentic artificial-intelligence system for the preliminary structural sizing of a transport-aircraft wing box. Rather than a hand or spreadsheet-driven design cycle, the system is organized as a directed graph of specialist computational and decision nodes, built with LangGraph [7], in which a large language model (LLM) is restricted to exactly the judgment calls a structural engineer would make, such as material family, spar location, stringer catalogue selection, web architecture and joint configuration, while every numerical quantity is produced by a deterministic, unit-tested Python implementation of the classical sizing methods of Niu [1, 2], Megson [3] and Roskam [4]. This is a project report, not a paper-reproduction study: the system is presented on its own terms, and an external published design is used once, as a validation benchmark, rather than as the report's own subject. The report first reviews the traditional, manual wing-box design process the system's phases are modeled on, then introduces the general concepts of agentic AI, large language models and the LangGraph orchestration framework, and describes how these are combined into the present system: its graph topology, the enforced separation between LLM decisions and Python computations, the material-selection method (an Ashby structural index combined with alloy-family screening and Pareto-front trade-off selection), and the governing equations used at each design phase. Implementation details are discussed next, followed by two verification benchmarks of deliberately different scope. The narrow one, a single cross-section worked by hand in the course lecture material, is reproduced to tabulated precision as an automated acceptance gate and surfaced four notational and arithmetic slips in the reference itself along the way. The broad one runs the complete pipeline, with the live LLM decider in every judgment node, against a real, independently published wing-box design (a Pilatus PC-21/Super Tucano-class trainer wing from a METU MSc thesis) and converges on a wingbox weight within about five percent of that design's own optimized result, corroborated by a real Abaqus finite-element solve of the converged geometry. A parameter table, acknowledgements and the reference list close the report.

1 Introduction

1.1 The wing box and the role of structural design

The wing box, the torsion-and-bending-carrying structure bounded by the front and rear spars, the upper and lower covers, and the ribs between them, is the primary load-bearing member of a fixed-wing aircraft. It reacts the spanwise bending moment and shear produced by lift and inertia, the torque produced by the offset between the aerodynamic center and the shear center, and it forms, in most transport designs, the integral fuel tank. Sizing it is a constrained optimization problem in the practical, not the mathematical, sense: minimum weight subject to strength, buckling stability, stiffness (flutter and divergence), fatigue and damage-tolerance, and manufacturability, under a load envelope that is itself a function of the structure's own weight. Niu [2] frames this as a genuinely iterative, multi-disciplinary negotiation, with the structural designer as the "focal point" between Loads, Materials, Weight and Balance, Fatigue, Manufacturing and several other groups, rather than a single closed-form calculation.

1.2 The classical design process

The traditional process this project's engine reproduces is documented across three source traditions that the aerospace-structures literature blends but does not always reconcile: Niu's own layout rules and allowables [1, 2], Roskam's spar-percentage and rib-pitch rules [4], and Megson's shear-flow notation for closed-cell, multi-boom sections [3]. It decomposes into nine phases, summarized in Table 1, each of which is a candidate unit of work in an automated pipeline.

Table 1: The nine phases of the classical wing-box design process.

Phase	Content
P0	Requirements: geometry, airworthiness basis, design load factors, candidate materials, wing-box-specific design conditions (thermal, fuel, crushing, hail, sonic fatigue, …).
P1	External loads: maneuver and gust load cases, spanwise lift and inertia distribution, integration to shear/moment/torque, the 1.5× ultimate factor (FAR 25.303).
P2	Geometric layout: spar chordwise location (Niu vs. Roskam disagree), rib pitch and direction, spar count, section height and enclosed area.
P3	Idealization: rectangular symmetric section, lumped-boom (skin-stringer) model, one governing load case.
P4	Cover sizing: required bending area, skin/stringer split, catalogue stringer selection, panel stability (Farrar efficiency, crippling, effective width, inter-rivet buckling), fail-safe design of the tension (lower) cover.
P5	Shear flow and web sizing: closed-cell shear flow (Megson), web thickness from strength and from buckling, shear-resistant vs. diagonal-tension architecture, spar caps.
P6	Ribs: crushing-load reaction, column-length limitation for the cover panels, concentrated-load redistribution, pitch-band check.
P7	Wing root joint and spanwise splices: joint-type trade-off, fastener count, bearing and shear allowables, fitting factor (FAR 25.625).
P8	Margins, stiffness, weight: margin of safety, EI/GJ stiffness for flutter, the non-optimum weight factor.

Two properties of this process motivate everything that follows. First, it is *cyclic, not linear*: rib pitch sets the unsupported column length in the Farrar equation, which sets skin thickness, which sets weight, which changes the load the rib pitch was chosen against. Niu's own "Structural Analytical Design Cycles" [2] describe exactly this loop, run two or three times against successive finite-element models. Second, *engineering judgment is intrinsic to the method, not a gap in it*: Niu repeatedly states that "the input of engineering judgment and/or assumption is frequently used to modify the conventional method of analysis to fit the need," and that under uncertainty "the level of conservatism is based upon engineering judgment." The classical process therefore already separates two kinds of work, applying a closed-form equation and choosing among several defensible options, even though a hand calculation does not enforce that separation explicitly. It is precisely this separation that an agentic system with a large language model in the loop can make explicit and auditable, which is the subject of the rest of this report.

2 Background: Agentic AI, Large Language Models, and LangGraph

2.1 Large language models and their decision-making role

A large language model (LLM) is a neural network, typically a transformer, trained to predict text conditioned on prior context, and subsequently adapted (instruction tuning, tool-calling training, structured-output training) to follow instructions and return machine-parseable results rather than free text alone. That training process exposes the model to an enormous volume of human-written technical reasoning, specification documents and worked decisions, which is what gives an instruction-tuned model its real, practically useful strength: weighing several qualitative or semi-quantitative criteria at once and producing a justified choice among them, in the same way Niu's own description of engineering judgment calls for (Section 1.2: "the level of conservatism is based upon engineering judgment"). Picking a material off a short, already-ranked, already-justified candidate list (Section 4), or trading a joint configuration's fail-safety against its manufacturing cost, is exactly this kind of decision: not a lookup, and not an arithmetic evaluation, but a bounded judgment call over criteria stated in the prompt.

What matters for an engineering application is less what an LLM *is* than what it is *reliable* at. Framing an open-ended choice against stated criteria, summarizing and citing unstructured source material, and producing output constrained to a declared schema are all strong, well-supported capabilities. Exact, multi-step arithmetic and precise recall of a numeric fact (a catalogue value, a physical constant) the model was not directly given are not: LLM output is unreliable there, a well-documented property rather than an implementation detail of any one model. A second, equally important limitation is that LLM output is *sampled*: the same prompt can legitimately return a different, still-valid decision on a repeated call, and the model carries no memory of a fact between separate calls unless that fact is placed back into the prompt context each time. This asymmetry, reliable judgment, unreliable arithmetic and recall, non-deterministic across repeated calls, is the starting point for the whole system design in Section 3: an LLM is used exactly where its reliability is strong and never where it is weak, and every decision it makes is bounded to a short, explicit list of options rather than an open-ended numeric answer.

Practically, this decision-making power is exposed through *structured output* (also called *tool calling* or *function calling*): the caller supplies a strict schema, here a Pydantic model with enumerated fields, a bounded numeric *count* field where the option space is small and checkable, and a free-text rationale, and the serving stack constrains the model's response to values that satisfy that schema. This is the mechanical hook the governing principle in Section 3 exploits: because the schema itself carries no field a computed quantity could be written into, the model's decision-making power is spent entirely on the enumerated choice and its justification, never on a number.

2.2 Accessing a hosted model: APIs and API keys

A model of the scale used in this project is not run locally inside the design pipeline; it is hosted on a remote server and reached over the network through an HTTP API, most commonly the "chat completions" request and response shape OpenAI popularized and that many other providers now also expose as an *OpenAI-compatible* interface. Every such request is authenticated with an *API key*, a secret credential tied to an account and, typically, to a metered, per-token billing arrangement. This has two direct consequences for the system's own design, not only for how it is deployed. First, a call to the live model is a real, authenticated, metered network operation, not a local computation, which is exactly why the graph's LLM decider is built as one swappable implementation behind a fixed interface (Section 6) rather than assumed always available. Second, it is why the deterministic scripted decider, which needs no key and makes no network call at all, is the system's actual regression baseline: it is the only decider guaranteed to run identically in any environment, including one with no key configured. Section 6 states which specific endpoint, key configuration and model this project uses for its own live runs.

2.3 Agentic AI systems

An *agent*, in the sense used throughout this report, is an LLM given access to external tools and a persistent state, run inside a control loop that lets it decide, call a tool, observe the result, and decide again. The LangChain documentation [6] draws a sharp, practically important distinction between two ways of building such a system:

- **Workflows**: the sequence of steps is predetermined by the system designer; the LLM (if any) is consulted at specific, fixed points.
- **Agents**: the LLM decides its own process dynamically: which tool to call, in what order, and when to stop.

Six documented workflow/agent patterns follow from this distinction, summarized in Table 2.

Table 2: Workflow and agent patterns [6].

Pattern	Structure
Prompt chaining	Sequential steps, each consuming the previous step's output; a predetermined path with optional conditional branches.
Parallelization	Independent sub-tasks fanned out and their results aggregated.
Routing	An input is classified and dispatched to one of several specialists.
Orchestrator-worker	An orchestrator splits a task into dynamically-sized sub-tasks, each processed independently and then synthesized.
Evaluator-optimizer	A generator's output is evaluated against criteria and looped until it is accepted.
Agent	An LLM in a tool-calling loop with no predetermined path, dynamic branching, and its own stopping condition.

A second, orthogonal set of *multi-agent* patterns governs how several agents (or specialist components) share, or deliberately do not share, context and state [8]: a stateless *router* that classifies and dispatches a request; *subagents*, where a main agent calls specialists as tools and each invocation restarts with an isolated, freshly-serialized context; *handoffs*, where control (and persistent state) transfers explicitly between agents; and *skills*, where one agent loads additional prompts or knowledge on demand without ever leaving its own context. The choice among these is not cosmetic: it determines whether a numeric fact computed by one specialist is reliably visible to the next, which is exactly the property a shared engineering design depends on (Section 3.2).

2.4 Agent orchestration frameworks, and why this project uses LangGraph

None of the ideas in Section 2.1 (a model, a tool, a loop) need any particular piece of software; several actively maintained frameworks package them differently, summarized in Table 3.

Table 3: Agent orchestration frameworks considered.

Framework	Control model
LangGraph [7]	A general state machine (`StateGraph`): typed nodes and edges over one shared, explicitly-reduced state; no built-in notion of a "conversation" between agents.
AutoGen [11]	Multiple chat agents exchange messages in a group conversation; state is implicit in the message history each agent sees.
CrewAI [12]	Role-based "crews" of agents, each with its own goal and backstory, coordinated through a manager agent or a fixed process.
OpenAI Agents SDK [13]	Lightweight agents that *hand off* a single active conversation thread to one another, one agent owning it at a time.
LlamaIndex Workflows [14]	Event-driven steps that emit and consume typed events, closer in spirit to a state machine than a conversation.
Semantic Kernel [15]	Plugins (tools) invoked by a planner or an orchestrating agent, built around a single assistant rather than a graph.

The framework choice is not interchangeable for this project, and the reason is the same one Section 3.2 argues at the architecture level: AutoGen, CrewAI and the OpenAI Agents SDK are all built around *conversational* agents, where an agent's own message history *is* its state, and coordination happens by passing messages or handing off an active thread between agents. That shape reproduces exactly the "stateless subagent" failure mode Section 3.2 identifies as wrong for this problem: a numeric fact one specialist produces (the layout node's spar placement, say) would have to be re-serialized into another agent's conversation for the next specialist to use it at all, trusting the model to copy it faithfully rather than a program to pass it directly. LangGraph is different in kind, not only in polish: its `StateGraph` is a general state machine with no built-in concept of a conversation at all, so a node can be "call a model" or "run a plain Python function" with no structural distinction between them, and a single shared, typed state (with an explicit, custom reducer for every field two nodes might write concurrently, Table 4) is threaded through every node whether or not it involves a model. That is what lets a specialist node in this project be *read the shared state, write back one typed field* rather than *converse with other agents about the state*, and it is what makes the parallel sizing fan-out and the evaluator-optimizer convergence loop, both already required by the classical design process itself (Section 1.2), native graph primitives (`Send`, a conditional edge) rather than behavior that would have to be simulated on top of a conversational protocol. LlamaIndex Workflows is the next closest in spirit, being event- rather than conversation-centric, but it does not offer LangGraph's specific combination of a Pregel-style parallel super-step model and pluggable, per-field reducers, both load-bearing in this project's own topology (Section 3.3).

2.5 LangGraph

LangGraph [7] is the orchestration framework used to implement the system. It describes itself as a low-level, unopinionated state-machine runtime for stateful, long-running agents, rather than an agent framework with a fixed control policy, the right level for a system whose process is mostly a known engineering procedure with a few, specific judgment points, not an open-ended search. Its core primitives, used throughout Section 3, are summarized in Table 4.

Table 4: LangGraph primitives used in this project [7, 9, 10].

Primitive	Role
`StateGraph(State)`	The graph container, parameterized by a shared, typed state schema.
Node	A plain function `state` → `partial state update`; may or may not call an LLM.
Edge / conditional edge	An unconditional transition, or one chosen at runtime by a router function reading the current state.
Reducer	The merge rule applied when two nodes write to the same state key in one super-step (e.g., the default *replace*, `operator.add`, or a custom merge such as this project's `upsert_margins`).
`Send`	Dynamic fan-out: spawn N parallel node instances, each with its own payload (the orchestrator-worker pattern).
`interrupt()`	A dynamic pause point for human review, resumable with `Command(resume=…)`.
Checkpointer	Persists a state snapshot per execution step, enabling crash recovery and "time travel" between design iterations.

Execution follows a Pregel-style super-step model: every node scheduled in the same super-step runs concurrently, and the run halts once no node is active and no message is in transit. This is what makes a parallel sizing fan-out (Section 3.2) essentially free to express and to execute.

3 System Architecture and Application

3.1 The governing design principle

The system is built to one governing rule, stated and enforced structurally rather than left to a prompt instruction:

> *The LLM never does arithmetic. Python never makes judgment calls.*

Every number that enters a design (a thickness, an area, a margin, a weight) is produced by a deterministic, unit-tested Python function implementing one of the equations in Section 5. Every *choice* among several defensible options (which spar-percentage convention to apply, which catalogue stringer to select, which joint configuration to use, which alloy family to draw from) belongs to an LLM decision node. Three independent, mechanically enforced layers make this falsifiable rather than aspirational:

1. **Decision schemas carry no numeric fields a computed quantity could be written into.** A cover-sizing decision, for instance, is constrained to an enumerated area-ratio basis, a catalogue section identifier, a stringer count, and a rationale with citations; there is no field in the schema an LLM could write a thickness or a stress into, even if it wanted to.
2. **Numbers enter the shared state only through a tool's return value, written by the node function that called it**, never directly by the LLM. A node's structure is always the same two-line shape: the decider is asked for a choice, and a Python sizing function is then called with that choice as one of its inputs.
3. **Every numeric quantity carries a provenance record** (which tool produced it, and which source citation the underlying formula comes from). A dedicated audit walks a finished design artifact and flags any numeric value without one; a design carrying such a value is reported infeasible rather than allowed to converge silently. Brief inputs (values the user directly supplies) are exempt from this only because they mark themselves as *given* rather than computed, so the audit can still tell an input value apart from an invented one.

3.2 Choice of topology

Section 2 named three general patterns; the system combines them deliberately rather than adopting any one alone. The nine design phases of Table 1 are a known, standardized procedure, so the graph backbone follows *prompt chaining*: the path is predetermined and audit-able, matching the certification expectation that a structural sizing procedure be reviewable step by step. Given a finished cover design, sizing the spar web, the ribs and the joints are mutually independent (each reads the cover's booms and the section geometry, but not each other's results), so those disciplines run as a *parallel fan-out*, one LangGraph super-step. Driving every margin of safety toward zero from below is an *evaluator-optimizer* loop: a purely deterministic checker builds the full margin table, and a rule-based router decides which upstream node, if any, can plausibly fix the worst failing margin, and sends the design back to it.

The system deliberately does *not* adopt an autonomous-agent topology at the top level, and does *not* adopt a supervisor-of-stateless- subagents topology either. The first choice follows directly from Section 1.2: the design sequence is documented, ordered and standardized, so letting an LLM decide the sequence itself would discard exactly the structure that makes the process auditable, for no benefit LangGraph's own guidance would call for here. The second choice rests on one specific, load-bearing fact about the subagent pattern noted in Section 2: subagent invocations are stateless, each one restarting with an isolated, re-serialized context. That is precisely wrong for this problem, whose defining difficulty is a *single shared design dossier*: the layout node's spar placement is read by the cover node, both are read by the web node, and all three are read by the checker. A stateless subagent boundary would force the entire design to be re-serialized into every specialist's prompt and would require trusting the LLM to copy numbers faithfully between calls, which is exactly the failure mode the governing principle exists to rule out. The system instead uses one `StateGraph` with a single shared, typed state, and specialist nodes that read and write it directly, realizing Niu's "designer as focal point" model with the checker/router pair playing that role over shared state rather than over message passing.

3.3 Graph topology

Figure 1 shows the conceptual backbone the system is built around: a linear intake and layout stage, a parallel sizing fan-out, and an evaluator-optimizer convergence loop before reporting.

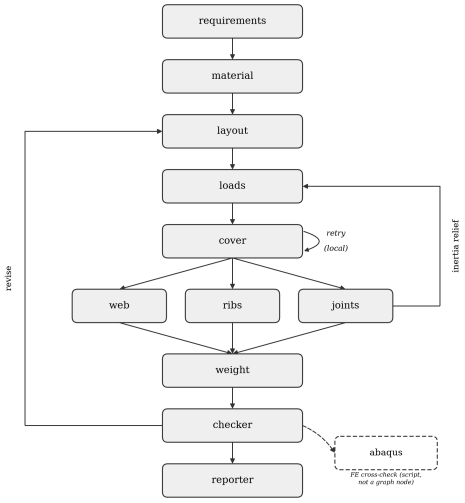

Figure 1: Conceptual graph backbone, with its two feedback loops (`revise`, `inertia relief`); the dashed `abaqus` box is a post-hoc script (Section 7), not a graph node.

The implemented graph elaborates this backbone considerably, because sizing a single cross-section (the scope of Figure 1) is only the starting point: a real wing box must be sized at every rib station along the span, under a real maneuver-and-gust load envelope, with splices, stringer run-outs and a root joint

placed and checked. The full topology, roughly thirty nodes, is

```
requirements → loads → material → layout →
rib_spacing → span → spanwise_loads → governing_loads
→ {cover | station_cover} → web → torsion → rib
→ rib_attachment → edge → geometry3d → splice →
station_cover → station_torsion → station_web →
checker → {layout | cover | plot} → discretize → runout
→ weight → spanwise_splice → joint → {inertia_relief
→ governing_loads (loop) | reporter}
```

Rather than reproduce every node individually, Table 5 groups them by the design responsibility they carry, which is the level at which the topology is actually meant to be read.

Table 5: The implemented graph, grouped by responsibility.

Group	Nodes
Intake & loads envelope	requirements, loads, rib_spacing, span, spanwise_loads, governing_loads
Material & layout	material, layout
Single-station sizing	cover, web, torsion, rib, rib_attachment, edge, joint
Spanwise generalization	geometry3d, splice, station_cover, station_torsion, station_web, spanwise_splice, discretize, runout
Verification & supervision	checker, weight
Outer convergence loop	inertia_relief (feeds back to governing_loads)
Visualization & reporting	plot, reporter

Two convergence loops coexist by design, at different scopes. The *inner* loop (`checker` → router-owner → ... → `checker`) drives every margin of a fixed load case toward MS ≥ 0. The *outer* loop (`joint` → `inertia_relief` → `governing_loads` → ...) re-derives the load envelope from the structure's own just-sized weight (the wing's mass opposing its own lift, per Niu [2]) and re-enters the inner loop against the corrected loads, until two successive weight estimates agree within a stated tolerance or a pass cap is reached. This is the computational form of the "preliminary sizing → resize" cycle Niu describes for a hand-driven process [2].

3.4 Decision versus computation: a representative sample

Table 6 lists a representative sample of nodes, split by whether they carry an LLM decision or are pure Python. The pattern is consistent throughout: wherever a node appears in both columns, it is because a Python-computed quantity is a precondition for an LLM's decision (for instance, the cover node's ranked material table, or the checker's margin table before the router chooses where to route a retry), never because the LLM computes anything itself.

3.5 Verification and the convergence loop

The `checker` node is, by design, the one node in the graph guaranteed to carry no model: it computes the margin of safety (Eq. (19)) for every applicable failure mode and component, audits every numeric quantity for provenance, and checks cross-equation consistency (unit dimensions; that a closed-cell shear-flow solution actually balances the applied shear and torque).

Each entry it produces is a frozen `Margin` record whose fields (Table 7) are what make the loop mechanical rather than interpretive. The `(component, mode)` pair is the routing key,

Table 6: Decision vs. computation, representative nodes.

Node	LLM?	What is decided / computed
requirements	no	Validates the brief, fills cited defaults, fails fast on an inconsistent input.
loads	no	Assembles the maneuver + gust V-n envelope from the brief's planform.
material	yes	Picks one alloy per cover position from a Python-ranked, family-screened, Pareto-front table (Section 4).
layout	yes	Chooses the spar-percentage convention (Niu vs. Roskam) and rib pitch/direction within cited bands.
governing_loads	yes	Selects which V-n cases enter the design envelope at every station (never a value, only case labels).
cover	yes	Chooses the area-ratio basis and the catalogue stringer section/count; Python sizes the required area and idealizes the section.
station_cover	no	Deterministic catalogue search: the lightest feasible pick clearing area, buckling and thickness constraints at every station.
web	yes	Chooses shear-resistant vs. diagonal-tension edge condition; Python solves Megson's shear flow and the governing thickness.
rib_spacing	yes	Chooses rib pitch/edge condition within Roskam's band; `rib` (later, pure Python) reads that choice back and computes crushing pressure and web sizing.
splice, runout	yes	Choose which stations carry a spanwise splice, and which side (front/rear spar) a stringer run-out sits on.
joint	yes	Chooses the root-joint configuration (spliced plates / tension bolts / lug / combined); Python searches the smallest feasible fastener count.
checker	no	Builds the full margin table and the provenance audit; deliberately model-free, since an arbiter that can hallucinate a passing margin is worse than none.
router	no	Rule-based lookup from a failing margin to the one node that can plausibly fix it (a fixed ownership table, not a judgment call).
weight, discretize	no	Spanwise integration of already-sized areas; fully-stressed-design reduction to a stepped, manufacturable geometry.
reporter	no	Template rendering of already-computed, already-cited structured state into a document.
abaqus	n/a	Not a graph node at all: `scripts/export_abaqus.py` builds and solves a real FE model from an already-finished design, run separately for the cross-check in Section 7.

so a failure names both the member that failed and the physical mode it failed in, which is precisely the granularity the ownership lookup needs: a `stringer_area` shortfall and a `chordwise_bending` shortfall are corrected by different nodes even though both are cover failures. Keeping `applied` and `allowable` as separate unit-carrying quantities, rather than only the resulting scalar, is what lets the retry feedback tell a decider *by how much* and *against what* its previous pick fell short instead of merely that it failed. `load_case` records which V-n case governed, so a margin computed from a spanwise envelope stays attributable to the one station and case that produced it after the worst-of-many reduction. And `provenance` names the Python function that produced the number, so the same audit that rejects an untraceable thickness applies to the margins judging it. Two derived predicates read off `value` directly: a negative margin fails, and one well above zero is flagged as wasted structural weight, since Niu's own design target is MS ≈ 0 rather than merely MS > 0.

The `router` reads that table and applies a fixed ownership lookup, choosing the single node that can plausibly correct a given `(component, mode)` failure, rather than asking an LLM to guess. A margin with no listed owner escalates directly to a `needs_human` verdict instead of retrying a node structurally unable to fix it. Two further guards prevent the loop from being an open-ended search: an iteration cap, so an unresolved

Table 7: Fields of a `Margin` record.

Field	Meaning
`component`	Which member: `cover_upper`, `cover_lower`, `web_front`, `web_rear`, `rib`, `spar_cap`, `stringer_upper/lower`, `joint_upper/lower`, `rib_joint`.
`mode`	Which failure mode: `stringer_area`, `min_thickness`, `column_buckling`, `combined_buckling`, `chordwise_bending`, `spanwise_bending(_reversed)`.
`applied`	The real applied stress or load, unit-carrying.
`allowable`	The allowable it is judged against, unit-carrying.
`value`	MS itself (Eq. (19)); <0 fails, $\gg 0$ is wasted weight.
`load_case`	Which V-n case governed, where one applies.
`provenance`	Which Python tool produced the number, for the audit.

design terminates rather than being left `running`; and a distinct `infeasible` outcome (a real "no design satisfies this brief" result, reserved for a defect no retry could fix, such as a provenance failure), kept separate from the ordinary no-progress case so the two are never conflated.

4 Material Selection

Material selection is treated as its own decision, run independently for the upper (compression-critical) and lower (tension-critical) cover, since Niu's own guidance on material usage differs by loading mode [2]. It is built as three Python-computed layers handed to the LLM decider in sequence, so that the model chooses only among options a deterministic calculation has already produced and ranked, never a number it estimates itself.

Candidate data. Two material sources feed the ranking. The first is an MMPDS-style handbook set carrying the small number of alloys (including the 2024-T3 card the reference worked example itself uses) needed to reproduce the acceptance-gate design exactly. The second, and by far the larger, is a database exported from ANSYS Granta CES Selector [17], a commercial materials-selection database and software platform widely used in mechanical and aerospace design for exactly this kind of ranking task. From Granta's own catalog, the candidate set was deliberately restricted to wrought aluminum alloys, the material class this project's all-metal wing box actually uses; every other class Granta reports (composites, steels, titanium, ceramics, and so on) is excluded from the search entirely, not merely down-ranked. That restriction yields 167 distinct alloy/temper rows, each carrying every property Granta reports for it, 189 columns in total, including price per pound and a qualitative weldability rating that the handbook set does not supply and that Layer 3 below depends on directly. Numeric properties originally reported as a range by Granta (typical for a commercial alloy/temper database, which reports the spread seen across product forms and thicknesses rather than one design value) are reduced to their midpoint before being used in the ranking, a conservative simplification stated here rather than silently folded into the numbers.

Layer 1: an Ashby structural index. Following Ashby and Cebon's method [16], a single figure of merit predicts which candidate material gives the lightest structure for a stated failure mode:

$$M_{\text{tension}} = \frac{F_{ty}}{\rho}, \tag{1}$$

$$M_{\text{buckling}} = \frac{E^{1/3}}{\rho}, \tag{2}$$

where F_{ty} is tensile yield strength, E is Young's modulus and ρ is density; higher M is better in both modes. Eq. (1) is the minimum-weight tie criterion, used for the lower (tension) cover; Eq. (2) is Ashby's standard index for a light, stiff, buckling-limited panel, used for the upper (compression) cover. Applied to the full 167-alloy Granta set plus the handbook rows, this index alone is computed over essentially the whole wrought-aluminum candidate pool at once.

Layer 2: an alloy-family screen. The raw index in Eqs. (1) and (2) is blind to fatigue and fracture behavior, and can rank an alloy with no service history in the relevant role above one real transports actually use. Niu [2] states plainly that 2024-series alloys dominate fatigue-critical (tension) applications while 7075-series alloys dominate high-compression construction; an independent transcription of 24 real transport wing-box material call-outs from the course's own reference data confirms this pattern almost without exception (7xxx upper cover, 2xxx lower cover). The candidate list, Granta rows included, is therefore screened to the alloy family the reference data actually shows for that cover position *before* ranking, so an index-favored but service-unproven alloy cannot reach the decision at all.

Layer 3: an exact Pareto front. Once cost and weldability enter the comparison, both reported directly by the Granta export described above, there is no longer a single best material: a cheaper, easier-to-weld alloy may cost some structural index, a genuine trade-off. Collapsing this to one weighted scalar (the Weighted Property Method, also a standard technique) would make that trade-off *for* the decider before it ever saw the options; instead the system computes the exact, pairwise-dominance Pareto front over $\{M, \text{cost}, \text{weldability}\}$ directly, since exhaustively ranking a candidate pool of this size on three axes is small enough to solve exactly. Every non-dominated point is handed to the LLM decider, whose returned rationale is required to state which axis its final choice spends on.

5 Governing Design Equations

This section collects the closed-form equations the system's Python tools implement, organized by design phase, in the same style Niu, Megson and this project's course material present them. Every equation below is implemented as an independently unit-tested function and carries a source citation at the point it is applied in the code; the notation follows Section 1.2's three traditions and is normalized in the parameter table (Table 13).

5.1 Geometry and idealization

The box height at the front and rear spar follows directly from the airfoil thickness distribution and the chosen spar chordwise

stations,

$$h_{\text{front}} = \left(\frac{t}{c}\right)_{\text{front}} c, \qquad h_{\text{rear}} = \left(\frac{t}{c}\right)_{\text{rear}} c, \tag{3}$$

and the box width is the chordwise distance between spars, $w = x_{\text{rear}} - x_{\text{front}}$. Following Niu's preliminary- sizing idealization [2], the real section is reduced to a rectangular, symmetric section with lumped booms carrying axial load and a skin carrying shear only (Megson's boom-skin idealization [3]); this yields the second moment of area I_{xx}, I_{zz} and the section centroid used throughout the remaining phases.

5.2 Cover sizing

For the symmetric two-boom idealization used here, bending stress and the required total cover area follow from $\sigma = Mc/I$ with $I = Ah^2/2$ and $c = h/2$ (the half-depth, since the covers straddle the neutral axis symmetrically), giving

$$A = \frac{M_x}{h\,\sigma_{\text{allow}}}, \tag{4}$$

where A is one cover's required cross-sectional area, M_x the applied bending moment and h the box height. This required area is then split between skin and stringer,

$$A = A_{\text{skin}}\left(1 + \frac{A_{\text{str}}}{A_{\text{skin}}}\right), \tag{5}$$

using one of three sourced ratios (Table 8) depending on the panel construction the layout/cover decision selects.

Table 8: Skin/stringer area ratios, Green 8.3.

Basis	$A_{\text{str}}/A_{\text{skin}}$
Equal buckling stress, skin & stiffener	1.4
Z-section, Euler column = skin buckling	1.5
Unflanged, integrally stiffened	1.7

Panel stability then governs the actual stringer selection. Farrar's efficiency factor [2] relates the panel's end load per unit width N, tangent modulus E_t and unsupported length L (the rib pitch) to the crippling stress f the optimum panel can sustain,

$$F = \frac{f}{\sqrt{NE_t/L}}, \qquad f = F\sqrt{NE_t/L}, \tag{6}$$

with F a tabulated efficiency factor by construction type (Z-, hat-, Y-stringer, or an optimum integral panel).

5.3 Stability: crippling, column buckling, panel buckling

The crippling stress of one corner element of a catalogue stringer section (Needham's method, closed form) is

$$\sigma_{cc} = K_e \frac{\sqrt{E\,F_{cy}}}{(b'/t)^{0.75}}, \qquad b' = \tfrac{1}{2}(a+b), \tag{7}$$

where a, b are the two leg widths meeting at the corner, t the local thickness, and K_e an edge-support coefficient (0.316 to 0.366 depending on how many of the corner's edges are free); the result is capped at F_{cy}, since a stocky corner squashes rather than buckles. The Johnson-Euler column check bridges long-column (Euler) and short-column (crippling-governed) behavior,

$$F_{\text{col}} = \frac{\pi^2 E}{(L'/\rho)^2}, \qquad L' = \frac{L}{\sqrt{c_{\text{fix}}}}, \tag{8}$$

with ρ the section's radius of gyration and c_{fix} an end-fixity coefficient (a distributed-load, pin-ended value is used for a stringer that picks up load continuously from the skin rather than at discrete end points). A cover panel under combined compression and shear is checked against the Bruhn/ICAS parabolic interaction curve,

$$R_L = \frac{\sigma}{F_{ccr}}, \quad R_S = \frac{\tau}{F_{scr}}, \quad \text{RF} = \frac{2}{R_L + \sqrt{R_L^2 + 4R_S^2}}, \tag{9}$$

evaluated against each load's real ultimate capacity (crippling/column for compression, F_{su} for shear) rather than the panel's own elastic initial-buckling stress, consistent with a skin designed to operate in the post-buckled regime. General elastic plate buckling and inter-rivet buckling both reduce to the same closed form,

$$F_{cr} = K\,\eta\,E\left(\frac{t}{b}\right)^2, \qquad F_{ir} = 0.9\,c\,E\left(\frac{t}{s}\right)^2, \tag{10}$$

with K a compression/shear buckling coefficient by edge condition and aspect ratio, η a plasticity-reduction factor, and (for inter-rivet buckling) s the rivet pitch and c a rivet-head fixity coefficient; Niu's printed constant 0.9 is the same K-to-k conversion used for F_{cr}, evaluated at Poisson's ratio $\mu = 0.3$.

5.4 Shear flow and web sizing

Following Megson's closed-cell method [3], the shear flow around an idealized boom-skin section is

$$q_s = -\frac{S_z}{I_{xx}} \sum_r A_r z_r + q_{s,0}, \tag{11}$$

where the summation runs over booms from an open cut and $q_{s,0}$ is the constant closing flow found by enforcing zero net twist rate around the cell (equivalently, Niu's cut method, which reaches the same result via $\sum M_y = 0$). Web thickness is governed by the more conservative of two checks, shear strength,

$$t_{\text{shear}} = \frac{1.5\,q_{\text{ave}}}{\tau_{\text{allow}}}, \qquad \tau_{\text{allow}} = 0.577\,F_{ty}, \tag{12}$$

and shear buckling,

$$\tau_{cr} = K_s E\left(\frac{t}{b}\right)^2 \;\Rightarrow\; t_{\text{buckling}} = \left(\frac{b^2 q}{K_s E}\right)^{1/3}, \tag{13}$$

with K_s a shear-buckling coefficient by panel aspect ratio and edge condition. In every reference case examined, Eq. (13) governs: a design consequence, not a coincidence, and the reason the system checks both rather than strength alone. Pure torsion

adds a constant flow $q = M_y/(2A_{\text{enclosed}})$; the torque itself is computed as the sum of the aerodynamic-center-to-shear-center lift offset,

$$T = S_z\,(x_{ac} - x_{sc}), \qquad x_{ac} = \tfrac{1}{4}c, \tag{14}$$

and, where a real chordwise camber line is available, the section pitching moment about the aerodynamic center from classical thin-airfoil theory, $C_{m,ac} = -\frac{\pi}{4}(A_1 - A_2)$, using the airfoil's own first two Fourier coefficients.

5.5 Ribs and crushing loads

A rib reacts the radial "crushing" load a curved, bent cover exerts inward on the structure. In the preliminary-design form (Niu Red Eq. 6.9.3), the crushing pressure follows from the section's own bending moment and stiffness alone, without a chordwise stress distribution,

$$p_{\text{crush}} = \frac{t_e\, h_c\, M_x^2}{2\,E\,I_{xx}^2}, \tag{15}$$

with t_e an equivalent smeared cover thickness and h_c the box depth; the running load a rib must react is then $w = p_{\text{crush}} \cdot s$ for a rib pitch s.

5.6 Joints and fasteners

A splice or root joint is sized from the axial force the design moment transfers across the covers, $F = M_x/h$, scaled first to ultimate load (FAR 25.303, ×1.5) and then by the fitting factor (FAR 25.625, ×1.15),

$$P_{\text{design}} = 1.15 \times (1.5 \times P_{\text{limit}}), \tag{16}$$

and checked against the governing (lower) of the fastener shear and bearing allowables, with a minimum practical fastener spacing of $4D$ (Green 8.6).

5.7 Loads: maneuver and gust envelope

The maneuver load-factor envelope follows the deck's own worked formulation: the stall boundary speed at load factor n inverts $n = C_{L\max}\, q\, S/W$ using the source's own equivalent-airspeed shortcut,

$$q = \frac{nW}{C_{L\max} S}, \qquad V_{\text{KEAS}} = \sqrt{296\,q}, \tag{17}$$

and the gust load-factor increment adds

$$\mu = \frac{2(W/S)}{\rho g \bar{c}\, C_{L\alpha}}, \qquad K_g = \frac{0.88\,\mu}{5.3 + \mu}, \qquad \Delta n = \frac{\rho S C_{L\alpha}}{2W}\, V\, U_g\, K_g, \tag{18}$$

where μ is Pratt's gust mass parameter, K_g the gust alleviation factor, and U_g the design gust velocity at the flight altitude.

5.8 Margin of safety and weight

Every check in the design reduces to the same margin-of-safety form (Niu Red 2.8),

$$\text{MS} = \frac{F_{\text{allow}}}{f_{\text{applied}}} - 1, \qquad \text{MS}_{\text{final}} = \min(\text{MS}_{\text{ult}}, \text{MS}_{\text{yield}}), \tag{19}$$

with a negative value failing and a large positive value flagged as wasted weight, since Niu's design target is MS $\approx$ 0. Total box weight adds a non-optimum factor to the ideally-necessary, directly sized weight,

$$W_{\text{total}} = W_{\text{ideal}}\,(1 + \text{NOF}), \qquad \text{NOF} \geq 0.50, \tag{20}$$

covering joints, cutouts, fatigue and fail-safe provision, minimum gages and corrosion allowance, structure the sizing tools above do not individually model.

6 Implementation

The system is implemented in Python, with the deterministic engineering core (`wingbox/`) kept strictly independent of the agent layer (`agents/`): every equation in Section 5 lives in the former as a pure function returning a unit-carrying, provenance-tagged `Quantity`, and is exercised by its own unit tests, generally against a worked numerical example transcribed from the source material, so that a formula is never trusted on inspection alone. The agent layer (`requirements`, `material`, `layout`, …, one module per node family) is built on LangGraph [7], with the shared design state expressed as a Pydantic model, the mechanism that lets every decision schema (Section 3) be validated structurally rather than by convention.

Two decider implementations satisfy the same interface. A *scripted* decider returns fixed, pre-chosen decisions and is used in the automated test suite to prove that the graph reproduces a known reference design's numbers exactly with no model in the loop at all, the direct, executable proof that the LLM is not computing anything, since replacing it with a fixed script changes nothing. A *live* LLM decider binds the same structured-output schemas to a chat model over an OpenAI-compatible endpoint (via `langchain-openai` [7]), currently DeepSeek-V3.2 [25] reached through the AvalAI proxy [24], chosen for reliable tool calling at low per-call cost, appropriate to a workload of short, schema-constrained decisions rather than open-ended reasoning. As Section 2.2 states in general, this connection is a real, authenticated network call: the endpoint base URL and an API key are read from environment configuration (a local `.env` file, kept out of version control) rather than written into source, so the same code runs against a different account, a different proxy, or no live connection at all (falling back to the scripted decider) with no code change. Each node may also be routed to its own model tier through that same environment configuration, so that higher-leverage nodes (layout, cover, the router) can use a stronger, costlier model than more procedural ones (web, ribs, joints) purely by naming a different model per node.

Verification is layered rather than left to a single test suite. A reference worked example transcribed from the course lecture material [5] serves as an acceptance gate that the deterministic core must reproduce, to tabulated precision, before anything downstream is considered meaningful (Section 7.1); every extraction of tabulated or digitized source data (the stringer catalogue, buckling coefficients, material allowables) ships with its own cross-check against a number the source itself computed. The provenance audit described in Section 3 runs as part of the

graph itself, not only as an offline test, so an untraceable number fails a real design run, not only a unit test. Static analysis (linting and strict type checking) is enforced across the codebase as a further, independent layer against the class of loud, immediately-detectable defects that a runtime test might still miss before it runs.

7 Verification Benchmarks

The system is verified against two independent benchmarks of deliberately different scope. The first is the course's own worked example, a single cross-section sized by hand in the lecture material, which fixes whether the deterministic core reproduces the classical method it implements at all. The second is a real, independently published wing-box design from an external MSc thesis, which asks the harder question of whether the full pipeline, with a live model in every judgment node, converges on a comparable structure for a whole wing. Both are run with the live `LLMDecider`; the first is additionally pinned as an automated acceptance gate that runs on every change.

7.1 The lecture worked example: a single cross-section

The narrowest benchmark is the wing-box cross-section worked through by hand in the course lecture material [5]: a 60 in chord NACA 2412 section carrying $V_z = 6{,}500$ lb and $M_x = 5.5 \times 10^5$ lb in, sized for spar placement, box height, cover area, skin/stringer split and spar-web thickness. Because every intermediate quantity is printed, this example fixes the deterministic core exactly rather than approximately, and it is wired into the test suite as a marked acceptance gate: nineteen assertions reproduce the reference's own numbers, and nothing downstream in the project is treated as meaningful until they pass.

Reconstructing it surfaced four notational and arithmetic slips in the reference itself, each identified only by reading the slide as an image (the text layer garbles every one of them) and each now pinned by a test so it cannot be silently "corrected" back. The printed box height 6.625 in is a digit transposition of 6.265 in, which is both what the NACA 2412 ordinates give at the two spar stations and what the same lecture uses correctly as the web depth b eight slides later; the bending depth and the web depth are one quantity, and a test asserts they stay so. The bending moment is substituted as 5.5×10^6 where the problem statement gives 5.5×10^5. The relation $I = 2Ah^2$ is stated symbolically but is valid only for h measured as the half depth: for a two-boom idealization separated by the full box height, $I = Ah^2/2$ and $c = h/2$, so $\sigma = Mc/I = M/(Ah)$ and Eq. (4) carries no factor of two; the slide's own arithmetic divides by $h\sigma$ and is correct, and only its symbolic statement is not. Finally the centroid offset $\bar{z} = -0.315/4.407$ is reported as -0.04 where it is -0.0715, harmless because the section is nearly symmetric either way. With the transposition and the symbolic error corrected and the problem statement's own moment used, the required cover area comes out at 2.194 in^2 against the printed 2.2.

Two design facts the example establishes are load-bearing for the rest of the system. The spar web is governed by buckling, not by shear strength: Eq. (12) gives $t = 0.055$ in where Eq. (13) gives 0.072 in, so an implementation checking only strength silently under-sizes the web, and `web_node` therefore sizes against the larger of the two by construction. And the upper cover is compression-critical under positive load factors while the lower is tension-critical, which is why `material_node` runs one independent ranking per cover position (Section 4) rather than choosing one alloy for the box.

Run through the full graph with the live `LLMDecider` in every judgment node, the same brief converges in a single iteration with all seven margins positive, the worst being cover column buckling at +70%. The model's own choices differ from the lecture's throughout and legitimately so: it places the spars at 15% and 58% chord rather than 16% and 55% (both inside the cited Niu band), and it selects 7249-T76511 for the compression-critical upper cover and 2195-T8 for the tension-critical lower cover in place of the lecture's single 2024-T3, both being the highest-structural-index rows on their own Pareto fronts. Its cover pick is a genuine two-section mix on the upper cover (2×#14 plus 1×#12 channel) against three of #14 on the lower, and Figure 2 draws that converged section with each stringer at its real catalogue profile. The whole run costs ∼8,000 tokens and under two minutes.

Table 9 sets the two designs side by side. The system agrees with the reference wherever the reference is a *constraint*, the same airfoil and chord, the same Niu spar band, the same sourced 1.4 area-ratio basis, the same conclusion that web thickness is buckling-driven, and departs from it wherever the reference was making a *choice*. The single largest departure is material: the lecture sizes both covers in 2024-T3 at a 40 ksi allowable, while the system's Ashby-index ranking and per-position Pareto screen select two different modern alloys at roughly twice that allowable, which by Eq. (4) halves the required cover area and cascades into a thinner skin and a much smaller stringer count. The two designs are therefore not two answers to the same arithmetic but two defensible points in the same design space, which is the intended behaviour: the deterministic core reproduces the reference exactly when handed the reference's own choices (that is the acceptance gate), and the agent layer is free to choose differently when it is not.

Table 9: The lecture worked example, sized by hand and by this system.

Quantity	Lecture (by hand)	This system (live LLM)
Front / rear spar	16% / 55% c	15% / 58% c
Box width	23.0 in	26.0 in
Box height h	6.265 in	6.019 in
Cover material, upper / lower	2024-T3 (both)	7249-T76511 / 2195-T8
Governing cover allowable	40.0 ksi (both)	81.3 / 79.6 ksi
Required cover area	2.194 in^2 (printed 2.2)	1.124 / 1.148 in^2
Skin/stringer area basis	1.4, equal-buckling	1.4, equal-buckling
Skin thickness	0.040 in	0.0180 / 0.0184 in
Stringer pick, per cover	#18×4, #27×5, #6×5 (7 per cover)	2×#14 + 1×#12 / 3×#14 (3 per cover)
Web: shear vs. buckling	0.055 / 0.072 in	0.019 / 0.078 in
Governing web mode	buckling	buckling
Column buckling	not checked by the source	+75% / +70%
Outcome	hand calculation	converged, 1 iteration, 7/7 margins pass

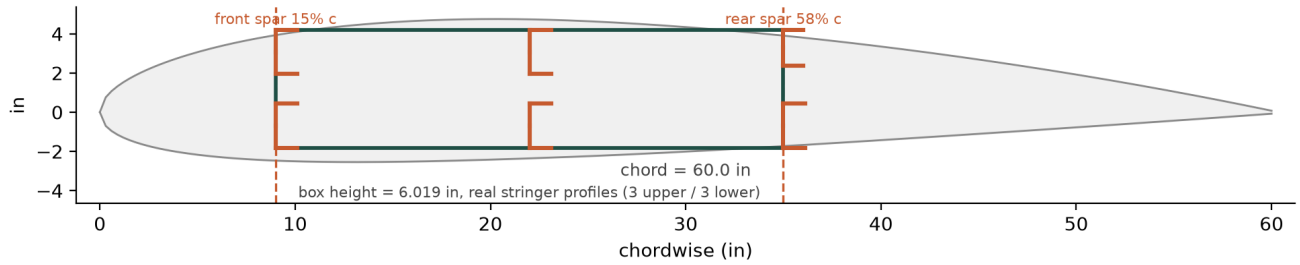

Figure 2: The lecture worked example's cross-section as converged by the live `LLMDecider`: NACA 2412 silhouette, box outline between the chosen spar stations, and every stringer drawn at its own real catalogue cross-section (channel sections #12 and #14). Leg lengths and chordwise positions are to scale; leg thickness is drawn at a fixed weight, a real 0.12–0.19 in gauge being imperceptible against a 60 in chord.

Two rows of that table deserve a caveat rather than a direct reading. The web thicknesses are computed from different shear flows: the lecture's 0.055 and 0.072 in follow from its own published panel flows, which this project independently found to violate vertical equilibrium by 10.7% (the reference mixes the stringer-centroid line for I_{xx} with the skin line for the enclosed area, and its web flows are off by exactly that same ratio), whereas the system's follow from a self-consistent closed-cell solve whose resultant reproduces the applied shear to machine precision. And the column-buckling row has no lecture entry to compare against because the reference never performs that check; running it against the reference's own slide-27 stringer pick, at this project's rib pitch, is what showed that pick would not survive it, which is a limitation of the worked example rather than a disagreement with it.

What makes that single-pass result possible is what the cover decider is shown before it answers. Alongside the stringer catalogue and each cover's required area, it receives that cover's real axial load and the column-buckling allowable of every candidate section, evaluated at the design's own rib pitch by the same crippling-plus-column-buckling routine (Eqs. (7) and (8)) that the checker node later uses to grade the result. The decider can therefore verify applied stress against the weakest allowable among its own picks before committing to them, rather than proposing a combination whose governing quantity it cannot evaluate. Every one of those numbers is computed in Python and merely shown to the model, so this strengthens the governing principle rather than relaxing it: the model still chooses only which sections and how many, and still computes nothing.

7.2 The METU PC-21 wing box

The broader benchmark runs the complete pipeline, real per-station loads, per-station cover/web/torsion sizing, splice and stringer run-out placement, the root joint, with the live `LLMDecider` in every judgment node, against Mert's MSc thesis at the Middle East Technical University [19], which develops a preliminary sizing tool for a Pilatus PC-21/Embraer Super Tucano-class turboprop trainer wing box and reports its own optimized result. The case-study script feeds the thesis's own real geometry, spar layout, material and station-by-station load table into the graph unchanged (Table 10), so the comparison isolates sizing methodology rather than posing a different problem.

Table 10: METU PC-21 wing box: inputs taken from the thesis, GIVEN provenance.

Quantity	Value (thesis source)
Root / tip chord	2258 mm / 1085 mm (Table 2-3)
Half span	4555 mm (Table 2-3)
Airfoil	NACA 2412 (Table 2-3)
Spar layout	3 spars at 10% / 40% / 75% chord (p. 84)
Material	2024-T851, all components (p. 84-85)
Rib count and stations	17, real non-uniform %-span locations (p. 84)
Section loads V, M, PM	given at all 17 stations (Table 4-2)

Everything else, the stringer catalogue picks (section and count), the resulting skin/web/stringer thicknesses, and the weight breakdown, comes from this project's own graph rather than from the thesis. The per-station cover node runs its deterministic catalogue search (Section 3) independently at each of the seventeen stations; because total weight is additive across stations for one fixed catalogue and area-ratio basis, this station-by-station greedy search already reaches the true minimum of its own discrete parametrization, with no outer optimization loop required. The search also enforces skin-panel local buckling, inverting Eq. (10) for the panel width at which F_{cr} equals the stringer's own crippling stress, alongside the stringer-level area and column-buckling checks it always ran.

Table 11 compares the resulting wingbox weight against the thesis's own reported figure. A first, naive comparison against the thesis's headline 132.5 kg suggests a near-exact match; that figure, however, is the thesis's own optimized wingbox weight *including* rib weight ($\approx$ 6.5 kg, 4.9% of the total, Table 4-5), while this project's own W_{ideal} (Eq. (20)) is defined, following [20]'s own Eq. 36, as skin, stringers, spar caps and webs only, with no rib term. The corrected, apples-to-apples row instead uses the thesis's own structure-only figure of 125.9 kg.

Table 11: METU wingbox weight, structure only (no ribs).

Design	Weight	vs. thesis
Thesis, optimized structure	125.9 kg	—
This project, live `LLMDecider`	131.9–132.4 kg	+4.9% to +5.1%

The residual ∼5% gap traces to two stated differences in parametrization rather than to a missing search: the thesis scales a continuous thickness distribution by two free coefficients un-

der a global weight-minimization search (grid exploration cross-validated with `fminsearch`), trading skin and spar-web thickness against each other freely, while this project's search draws from a fixed table of 40 real extruded stringer sections [5] under one fixed, sourced skin/stringer area-ratio basis (Section 4). A discrete catalogue and an uncoupled area-ratio basis are both real engineering constraints the thesis's own continuous, coupled parametrization does not carry, so a small residual gap is the expected outcome of comparing two legitimate but different parametrizations, not a discrepancy left to close further.

7.3 Finite-element cross-check

The export tool builds a real shell/truss Abaqus 2024 model directly from each converged design, the project's own reference planform and the METU case, both with the live `LLMDecider` in the loop, and solves it, so the FE cross-check runs against the exact converged state rather than a separately re-built model. Table 12 reports two independent checks read back from each solved job: whether the root reaction balances the applied spanwise shear (an equilibrium check the analytical sizing never directly enforces at the FE mesh level), and how many of the truss stringer elements' own solved axial stress exceed the analytical crippling-plus-column-buckling allowable [2] already computed for the same pick by the checker node.

Table 12: Abaqus FE cross-check, three converged designs.

Design	Root reaction vs. applied	Crippling FAILs
Reference planform	balances to ∼0.00%	4 of 8 runs
METU, live `LLMDecider`	−27,426.7 vs. +27,426.7 lb	0 of 6 runs

Figure 3 shows the solved METU model, deformed shape contoured on displacement magnitude and rendered at a uniform deformation scale (a stated visualization choice, applied to the plotted shape only, so the real bending pattern is visible at a wing's real span-to-deflection ratio), root encastre'd at bottom left, tip visibly deflecting under the applied station loads. A FAIL entry in Table 12 does not by itself report a new finding: every truss element's cross-sectional area is drawn from the root design's own single-station stringer pick carried outboard by the layout rule (Section 3), not from the real per-station search, so a FAIL mostly corroborates a pre-existing, already-known limitation of that construction rather than a new defect in either the FE export or the analytical allowable.

7.4 An independent-judgment experiment

Every result above fixes material, spar layout and rib pitch to the thesis's own real, published values, isolating sizing methodology as the thing under test. A separate run instead gives the layout, material and rib-spacing deciders only the real geometry and loads a design engineer would start from, plus a free-text reference-aircraft hint, withholding the thesis's own numeric spar percentages, material and rib pitch, and asks whether the system's own judgment layer converges on something similar to what was actually built. The LLM correctly infers, from the qualitative aircraft-family hint alone, that a three-spar layout is warranted, matching the real aircraft's own published structural arrangement. Its other choices diverge substantially (front spar at 15.0% vs. the thesis's 10.0%, rib pitch 30.0 in vs. an ∼11.2 in real average, material 7249-T76511 vs. 2024-T851), producing a materially narrower box (38.0 in vs. 57.5 in) whose 124.5 kg result is consequently not directly comparable to Table 11: it sizes a different wingbox, not the same one sized differently. This confirms the judgment layer applies its own general, sourced preliminary-design rules (Niu/Roskam bands, Ashby-index material ranking) correctly and consistently, rather than reproducing the accumulated, unstated engineering choices, fuel-bay layout, actuator pass-throughs and manufacturing practice that one real, certified aircraft's own rib spacing happens to encode.

8 Conclusion

This report has described a working agentic system for preliminary wing-box structural design: a deterministic engineering core implementing the classical hand-calculation methods of Niu, Megson and Roskam, and an agent layer, built on LangGraph, that restricts a large language model to exactly the judgment calls those methods leave open, under a governing principle enforced by schema design, tool-only numeric entry, and a mechanical provenance audit rather than by prompt instruction alone. The system runs end to end, with a live model in the decision loop, and converges on the project's own reference planform. Section 7 validates it at two scales. At the narrow scale, the course's own hand-worked cross-section is reproduced to tabulated precision as an automated acceptance gate, and the live model, given the same brief, converges in one iteration with every margin positive while making defensible choices of its own, different spar percentages inside the same cited band and a separate alloy per cover position in place of the lecture's single one. Running that benchmark live also exposed and closed a real gap in the system: a decider asked to satisfy a column-buckling constraint whose governing allowable it had never been shown, now given the same per-section numbers the checker node already computes. At the broad scale, the full pipeline, run with the live LLM decider against a real, independently published wing-box design, the METU PC-21 case study, converges on a wingbox weight within about five percent of that design's own optimized result, with a real Abaqus finite-element solve corroborating static equilibrium at the converged design. The residual gap traces to stated, legitimate differences in parametrization (a discrete stringer catalogue and an uncoupled area-ratio basis, against the thesis's own continuous, coupled two-coefficient search), not to an error in either sizing method, and the independent-judgment experiment of Section 7.4 shows the system's own material/layout/rib-pitch judgment correctly inferring a qualitative structural arrangement from a reference-aircraft description and applying its own general, sourced design rules consistently across every case tested. Together, the two benchmarks show a system whose decision layer and computational core each do exactly the job the governing principle of Section 3 assigns them.

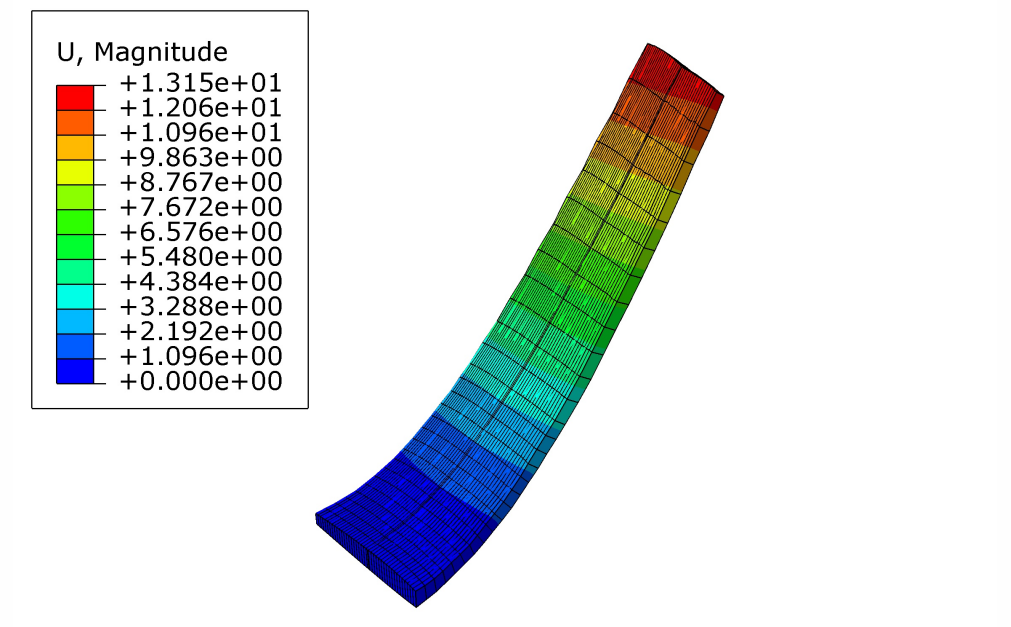

Figure 3: Solved Abaqus 2024 model of the METU PC-21 wingbox, deformed shape contoured on displacement magnitude and plotted at a uniform deformation scale so the bending pattern is visible at a wing's real span-to-deflection ratio; root at bottom left (encastre'd), tip at top right.

Table 13: Nomenclature and parameters used throughout this report.

Symbol	Meaning	Symbol	Meaning
A	Total required cover cross-sectional area	$A_{\text{skin}}, A_{\text{str}}$	Skin / stringer area split
A_r, z_r	Boom area / coordinate (idealized section)	a, b	Corner leg widths (Needham crippling)
b, b'	Panel short side / Needham mean leg width	c (chord)	Local wing chord
c (fixity)	End-fixity coefficient (column / rivet)	$\bar{c}$	Mean aerodynamic chord
$C_{L\max}, C_{L\alpha}$	Maximum lift coefficient / lift-curve slope	$C_{m,ac}$	Pitching moment about the aerodynamic center
D	Fastener diameter	E, E_t	Young's modulus / tangent modulus
F	Farrar efficiency factor	F_{allow}	Allowable stress/load
$F_{cc}, F_{cy}, F_{ty}, F_{su}$	Crippling / compressive yield / tensile yield / ultimate shear allowable	F_{cr}, F_{ir}	Plate / inter-rivet buckling stress
f, f_{applied}	Crippling / applied stress	h, h_c	Box (bending) depth
I_{xx}, I_{zz}	Second moment of area about x, z	K_c, K_s	Compression / shear plate-buckling coefficient
K_e	Needham corner edge-support coefficient	K_g	Gust alleviation factor
L, L'	Panel / effective column length	M (Ashby)	Ashby structural performance index
M_x, M_y, M_z	Bending / torsion / chordwise moment	MS	Margin of safety
N	Panel end load per unit width	NOF	Non-optimum weight factor
n, n_1, n_2, n_3	Load factor / category-specific load factors	$P_{\text{limit}}, P_{\text{design}}$	Limit / design (ultimate×fitting) load
$q, q_s, q_{s,0}$	Dynamic pressure / shear flow / closing shear flow	q_t	Torsional shear flow
RF	Combined-buckling reserve factor	R_L, R_S	Compression / shear stress ratios
S_z	Applied transverse shear	s	Rivet pitch / rib spacing
T	Applied torque	$t, t_{\text{skin}}, t_{\text{str}}$	Thickness (general / skin / stringer)
t/c	Airfoil thickness ratio	U_g	Design gust velocity
V, V_{KEAS}	Airspeed (general / knots equivalent)	w	Box width
W	Aircraft or wing-box weight	x_{ac}, x_{sc}	Aerodynamic-center / shear-center chordwise location
ρ (material)	Material density	ρ (section)	Radius of gyration
σ, τ	Normal / shear stress	σ_{cc}	Crippling stress
η	Plasticity reduction factor	μ (gust)	Gust mass parameter

Acknowledgements

I would like to express my sincere gratitude to Dr. Haddadpour for his instruction and guidance throughout the *Aircraft Structural Design* course, whose lecture material forms the process backbone this system implements, and to Eng. Samiei for teaching-assistant support throughout the term. The knowledge I gained from both underlies this work, and I intend to build on it in the stages ahead.

References

[1] C. Y. (Michael) Niu. *Airframe Structural Design: Practical Design Information and Data on Aircraft Structures.* Conmilit Press, 1988. Referred to throughout as the "Green Book"; source of the wing-box layout, cover and root-joint design rules (Ch. 8).

[2] C. Y. (Michael) Niu. *Airframe Stress Analysis and Sizing.* Conmilit Press, 2nd ed., 1997. Referred to throughout as the "Red Book"; source of the sizing procedures, margin-

of-safety definition, crippling, column-buckling, crushing-load and non-optimum-weight formulas used in Section 5.

[3] T. H. G. Megson. *Aircraft Structures for Engineering Students.* Butterworth-Heinemann. Source of the closed-cell, multi-boom shear-flow notation (Eq. (11)) used throughout the web-sizing tools.

[4] J. Roskam. *Airplane Design.* DARcorporation. Source of the spar chordwise placement and rib-pitch rules used as an alternative to Niu's own convention.

[5] H. Haddadpour. *Aircraft Structural Design*, lecture notes, decks 2 to 9. Department of Aerospace Engineering, Sharif University of Technology. Unpublished course material; source of the reference worked example used as this project's acceptance gate, the V-n and gust-envelope formulation, and the layout worked example this report reproduces in Table 1.

[6] LangChain. *Workflows and agents.* `https://docs.langchain.com/oss/python/langgraph/workflows-agents`. Source of the workflow/agent pattern taxonomy in Table 2.

[7] LangChain. *LangGraph Graph API.* `https://docs.langchain.com/oss/python/langgraph/graph-api`. Source of the `StateGraph`, node, edge, reducer, `Send` and compilation primitives in Table 4.

[8] LangChain. *Multi-agent architectures.* `https://docs.langchain.com/oss/python/langchain/multi-agent`. Source of the router/subagent/handoff/skill pattern comparison behind the topology choice in Section 3.2.

[9] LangChain. *Interrupts / human-in-the-loop.* `https://docs.langchain.com/oss/python/langgraph/interrupts`. Source of the `interrupt()` / `Command(resume=…)` mechanism referenced in Table 4.

[10] LangChain. *Persistence.* `https://docs.langchain.com/oss/python/langgraph/persistence`. Source of the checkpointer / thread model referenced in Table 4.

[11] Microsoft. *AutoGen.* `https://microsoft.github.io/autogen/`. A conversational multi-agent framework, compared in Table 3.

[12] CrewAI, Inc. *CrewAI.* `https://docs.crewai.com`. A role-based multi-agent framework, compared in Table 3.

[13] OpenAI. *OpenAI Agents SDK.* `https://openai.github.io/openai-agents-python/`. A handoff-based agent framework, compared in Table 3.

[14] LlamaIndex. *Workflows.* `https://docs.llamaindex.ai/en/stable/module_guides/workflow/`. An event-driven agent orchestration layer, compared in Table 3.

[15] Microsoft. *Semantic Kernel.* `https://learn.microsoft.com/en-us/semantic-kernel/`. A plugin/planner-based agent framework, compared in Table 3.

[16] M. F. Ashby. *Materials Selection in Mechanical Design.* Butterworth-Heinemann. Source of the structural performance index method in Section 4, Eqs. (1) and (2).

[17] ANSYS, Inc. *ANSYS Granta CES Selector.* `https://www.ansys.com/products/materials/granta-selector`. Materials-selection database and software; source of the 167-alloy wrought-aluminum candidate set (189 reported properties per row, including cost and weldability) used in Section 4.

[18] Federal Aviation Administration. *Federal Aviation Regulations Part 25, Airworthiness Standards: Transport Category Airplanes.* §25.303 (factor of safety) and §25.625 (fitting factor); source of the 1.5× ultimate and 1.15× fitting factors in Eq. (16).

[19] M. Mert. *A Preliminary Sizing Tool for Minimum Weight Aircraft Wingbox Structural Design.* MSc thesis, Middle East Technical University, 2018. Source of the Pilatus PC-21/Embraer Super Tucano-class trainer wing box, its geometry, spar layout, material and station load table, used as the external validation case study of Section 7.

[20] A. Elham, G. La Rocca, M. J. L. van Tooren. *Development and implementation of an advanced, design-sensitive method for wing weight estimation.* Aerospace Science and Technology, 29(1):100-113, 2013. Source of the wing torque decomposition and whole-wing weight regression used in the spanwise torsion and weight-estimation tools.

[21] F. W. Diederich. *A Simple Approximate Method for Calculating the Spanwise Lift Distribution and Aerodynamic Influence Coefficients at Subsonic Speeds.* NACA Technical Note 2751, 1952. Source of the wing lift-curve-slope closed form used in the spanwise loads tool.

[22] I. McNaughtan. *Bird Impact on Aircraft Structures.* RAE Technical Report 72056, 1972. Source of the leading-edge bird-strike minimum-thickness criterion.

[23] C. Kassapoglou. *Design and Analysis of Composite Structures: With Applications to Aerospace Structures.* 2nd ed., Wiley, 2013. Ch. 7 (Post-Buckling); source of the framing for grading a post-buckled panel against its ultimate rather than its initial-buckling capacity, used in Eq. (9).

[24] AvalAI. *OpenAI-compatible model API proxy.* `https://avalai.ir`. The inference endpoint through which the live LLM decider is reached, via `langchain-openai`.

[25] DeepSeek. *DeepSeek-V3.2.* The chat model used for the live decision nodes described in Section 6, selected for reliable structured-output tool calling at low per-call cost.